

LTL Model Checking

(Ch. 4 LN)

Wishnu Prasetya

wishnu@cs.uu.nl

www.cs.uu.nl/docs/vakken/pv

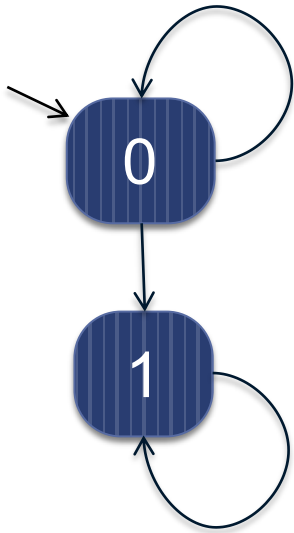
Overview

- This pack :
 - Abstract model of programs
 - Temporal properties
 - Verification (via model checking) algorithm
 - Concurrency

Run-time properties

- Hoare triple : express what should hold when the program terminates.
- Many programs are supposed to work continuously
 - They should be “safe”
 - They should not dead lock
 - No process should starve
- Linear Temporal Logic (LTL)
 - Originally designed by philosophers to study the way that time is used in arguments
Based on a number of operators/modalities to express relation over time: “next”, “always”, “eventually”
 - Belong to the class of modal logics
 - Brought to Computer Science by Pnueli, 1977.

Finite State Automaton/Machine



- Abstraction of a real program
- Choices
 - What information do we want to put in the states?
 - In the arrows?
- How to model execution? → a path through the FSA, starting at an initial state.
 - Does it have to be finite?
 - Do we need a concept of “acceptance” ?
- These choices influence what you can express, and how you can verify properties over executions.

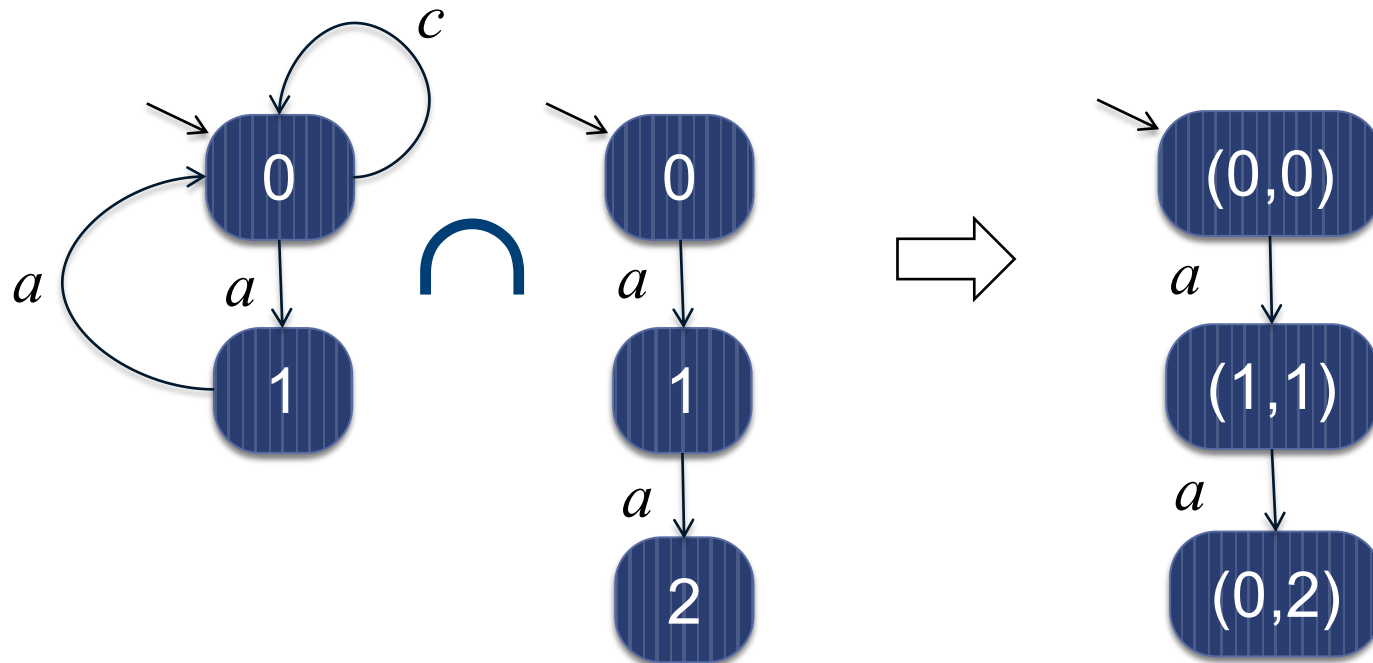
FSA

- Described by (S, I, Σ, R)
 - S : the set of possible states
 - $I \subseteq S$: set of possible initial states
 - Σ : set of labels, to decorate the arrows.
 - They model possible actions.
 - $R : S \rightarrow \Sigma \rightarrow \mathbf{pow}(S)$, the arrows
 - $R(s, a)$ is the set of possible next-states of a if executed on s
 - non-deterministic

Program compositions can be modeled by operations over FSA

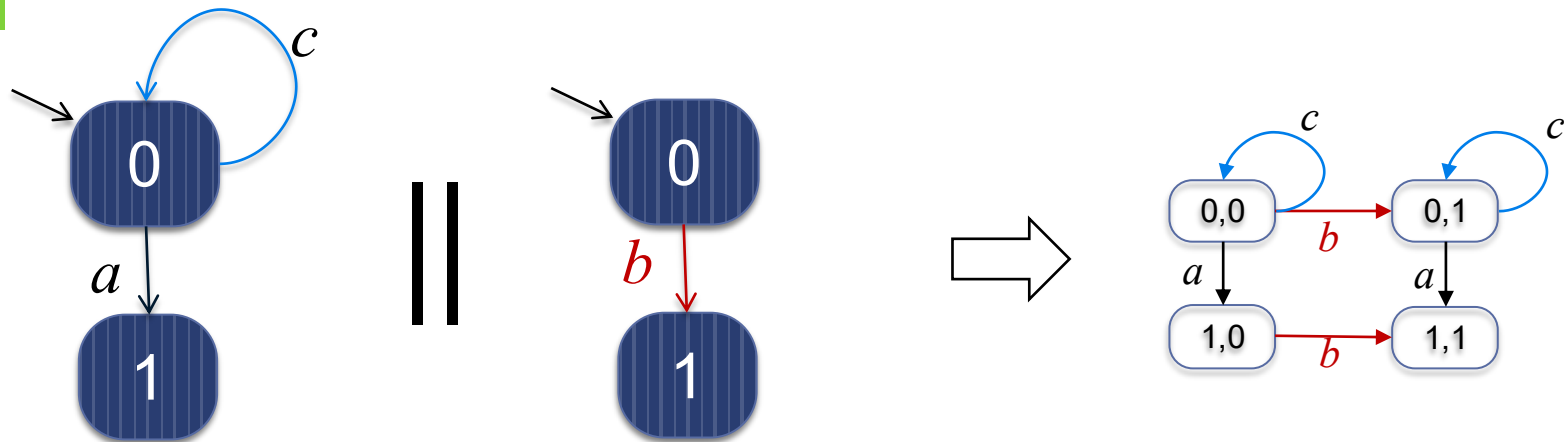
- We assume actions to be **atomic**.
- $M_1 ; M_2$
 - connect “terminal states” of M_1 to M_2 ’s initial states.
- $M_1 \cap M_2$
 - only do executions that are possible in both
- $M_1 \parallel M_2$
 - model parallel execution of M_1 and M_2

Intersection



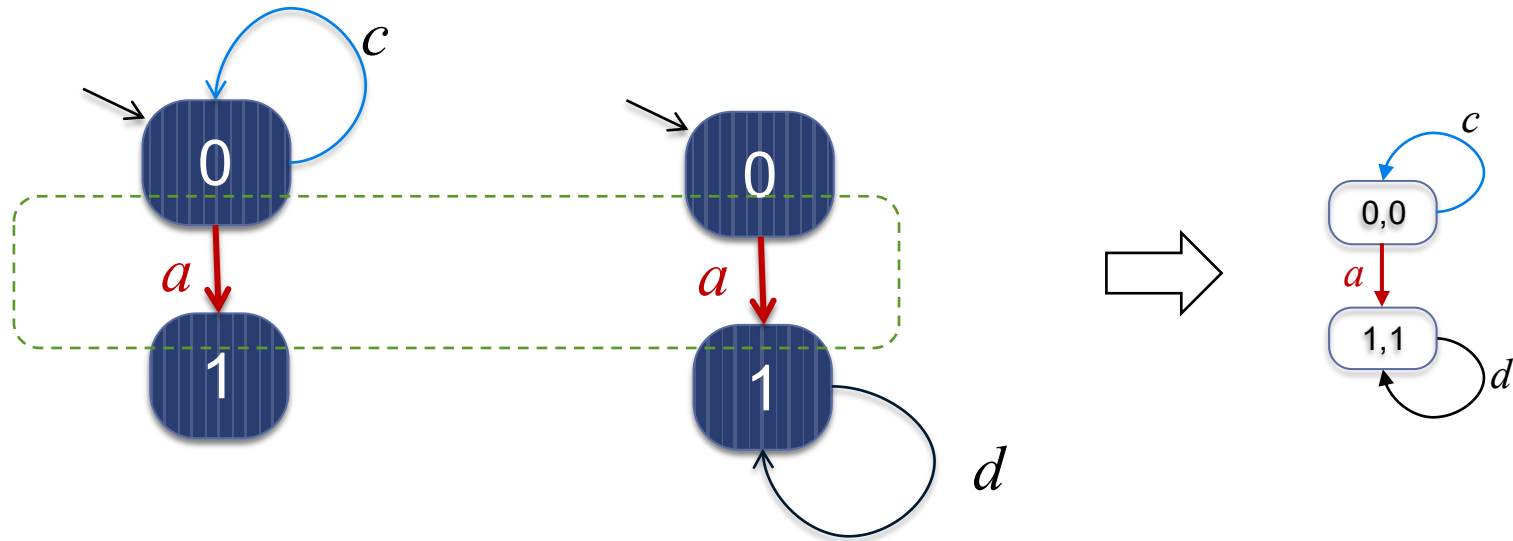
- Not something you typically do in real programs
- A useful concept for verification $\rightarrow$ later.

Parallel composition



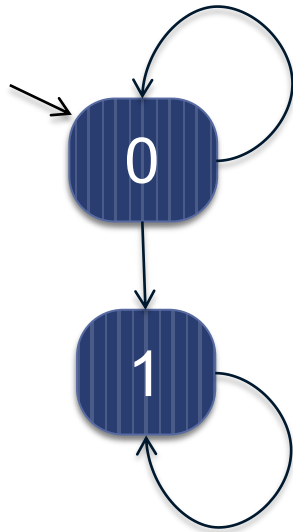
- Suppose M_1 and M_2 has **no common action** (no action on which they want to synchronize)
- Their parallel composition is basically the “full product” of M_1 and M_2 .
- So, if each component has n number of states, constructing $\parallel$ over k components produces an automaton with n^k states.

Parallel composition with synchronized actions



- Suppose we require that any action $a \in \Sigma_1 \cap \Sigma_2$ has to be executed **together** (synchronizedly) by both automata.

Let's add state-labels



Consider these set of “labels”,
Prop = { isOdd x , $x > 0$ }. The
labeling is describe by this function
 V :

$$V(0) = \{ \text{isOdd } x \}$$

$$V(1) = \{ \text{isOdd } x, x > 0 \}$$

So far, the only things we know about the states is that they differ from each other. Let's extend the available information with propositions, expressing properties that we know to hold in each state.

Kripke Structure

- A finite automaton ($S, s_0, R, Prop, V$)
 - S : the set of possible states, with s_0 the initial state.
 - $R : S \rightarrow \mathbf{pow}(S)$, the arrows
 - $R(s)$ is the set of possible next-states from s
 - non-deterministic
 - $Prop$: set of *atomic* propositions
 - abstractly modeling state properties.
 - $V : S \rightarrow \mathbf{pow}(Prop)$, labeling function
 - $a \in V(s)$ means a holds in s , else it does *not* hold.
 - No concept of accepting states.

Prop

- It consists of *atomic* propositions.
- We'll require them to be non-contradictive. That is, for any subset Q of $Prop$ the conjunction:

$$\bigwedge Q \wedge \bigwedge \{\neg p \mid p \in Prop/Q\}$$

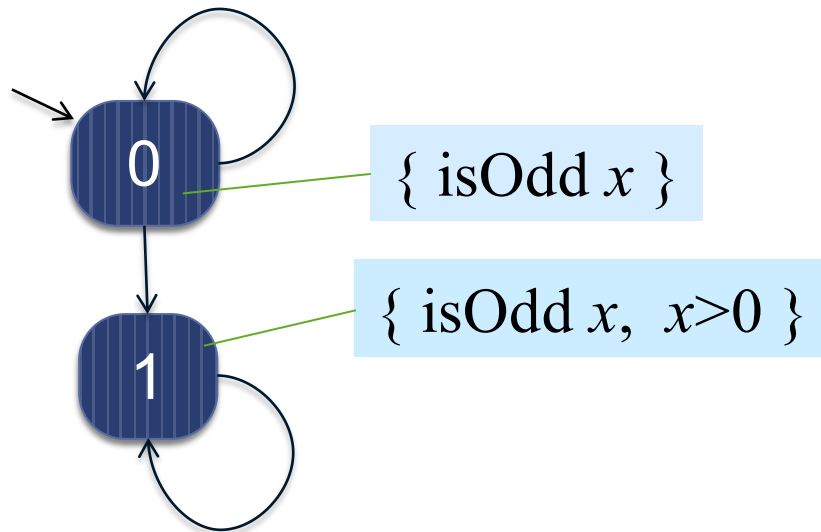
is satisfiable. This is the case if they are *independent* of each other.

- Else you may get inconsistent labeling.
- Example:
 - $Prop = \{ x > 0, y > 0 \}$ is ok.
 - $Prop = \{ x > 0, x > 1 \}$ is not ok. E.g. the subset $\{ x > 1 \}$ is inconsistent.

Execution

- An *execution* is a path through your automaton, starting from an initial state.
- Let's focus on properties of infinite executions
 - All executions are assumed **infinite**
 - Extend each “terminal” state (states with no successor) in the original Kripke with a stuttering loop.
- This induces an ‘*abstract*’ execution: $\text{Nat} \rightarrow \mathbf{pow}(\text{Prop})$
 - infinite *sequence of the set of propositions* that hold along that path.
 - We'll often use the term execution and abstract execution interchangeably.

Example

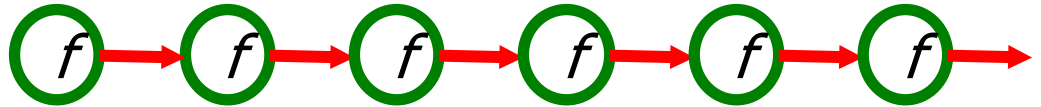


execution : **0** **0** **1** **1** ...

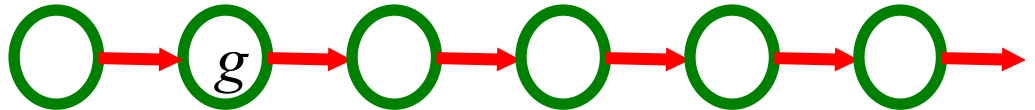
abstract
execution : $\{ \text{isOdd } x \}$, $\{ \text{isOdd } x \}$, $\{ \text{isOdd } x, x > 0 \}$, $\{ \text{isOdd } x, x > 0 \}$, ...

LTL, informal meaning

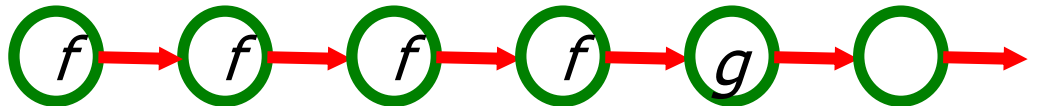
$\Box f$ // always f



$X g$ // next g



$f U g$ // f holds
until g



You can combine operators

- $\Box(p \rightarrow (\text{true } \mathbf{U} q))$ // whenever p holds, eventually q will hold
- $p \mathbf{U} (q \mathbf{U} r)$
- $\text{true } \mathbf{U} \Box p$ // eventually stabilizing to p
- $\Box(\text{true } \mathbf{U} p)$ // p holds infinitely many often

Syntax

- $\varphi ::= p$ // atomic proposition from *Prop*

| $\neg\varphi$ | $\varphi \wedge \psi$ | $X\varphi$ | $\varphi U \psi$

- Derived operators:
 - $\varphi \vee \psi = \neg(\neg\varphi \wedge \neg\psi)$
 - $\varphi \rightarrow \psi = \neg\varphi \vee \psi$
 - $\Box, \Diamond, W, \dots$
- Interpreted over abstract executions.

Defining the meaning of temporal formulas

- First we'll define the meaning wrt to a single abstract execution. Let Π be such an execution:
 - $\Pi, i \models \varphi$
 - $\Pi \models \varphi \quad = \quad \Pi, 0 \models \varphi$
- If P is a Kripke structure,

$P \models \varphi$ means that φ holds on all abstract executions of P that start from P 's initial state

Meaning

- Let Π be an (abstract) execution.

- $\Pi, i \models p \quad = \quad p \in \Pi(i) \quad // \quad p \in Prop$

- $\Pi, i \models \neg\varphi \quad = \quad \text{not } (\Pi, i \models \varphi)$

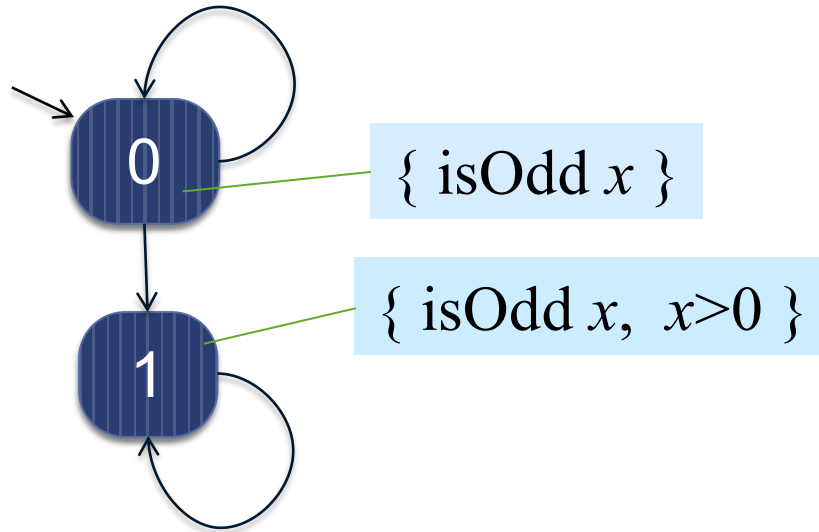
- $\Pi, i \models \varphi \wedge \psi \quad = \quad \Pi, i \models \varphi$
and
 $\Pi, i \models \psi$

Meaning

- $\Pi, i \models \neg \varphi = \Pi, i+1 \models \varphi$
- $\Pi, i \models \varphi \cup \psi =$ there is a $j \geq i$ such that $\Pi, j \models \psi$
and
for all $h, i \leq h < j$, we have $\Pi, h \models \varphi$.

Example

P:



Consider $\Pi : \{ \text{isOdd } x \}, \{ \text{isOdd } x \}, \{ \text{isOdd } x, x > 0 \}, \{ \text{isOdd } x, x > 0 \}, \dots$

$\Pi \models \text{isOdd } x \cup x > 0$

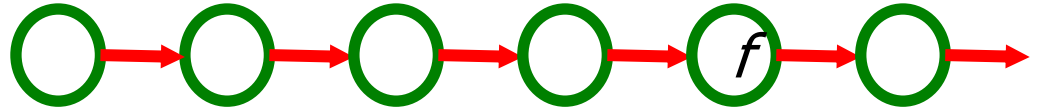
Is “ $\text{isOdd } x \cup x > 0$ ” also a valid property P ?

Derived operators

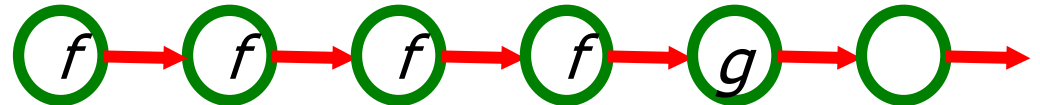
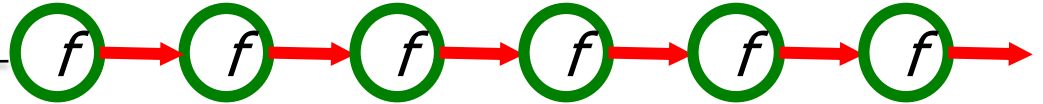
- Eventually $\diamond\varphi = \text{true } \mathbf{U} \varphi$
- Always $\Box\varphi = \neg\diamond\neg\varphi$
- Weak until $\varphi \mathbf{W} \psi = \Box\varphi \vee (\varphi \mathbf{U} \psi)$
- Release $\varphi \mathbf{R} \psi = \psi \mathbf{W} (\varphi \wedge \psi)$

Some derived operators

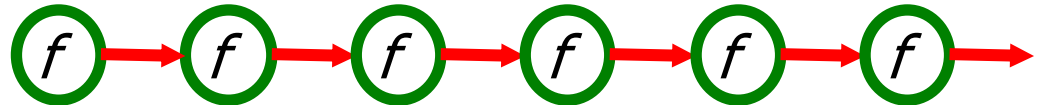
$\langle \rangle f$ // eventually f



$f W g$ // weak until



$g R f$ // releases



Past operators

- Useful, e.g. to say: if P is doing something with x , then it must have been given a permission to do so.
- “previous”
 $\Pi, i \models Y \varphi \quad = \quad \varphi \text{ holds in the previous state}$
- “since”
 $\Pi, i \models \varphi \text{ S } \psi \quad = \quad \Pi, j \models \psi \text{ for some } j \leq i, \text{ and for all } j < k \leq i \text{ we have } \Pi, j \models \varphi$
- Unfortunately, not supported by SPIN.

Ok, so how can I verify $M \models \varphi$?

- We can't directly check all executions $\rightarrow$ infinite (in two dimensions).
- Try to prove it directly using the definitions?
- We'll take a look another strategy...
- First, let's view abstract executions as *sentences*.

View M as a sentence-generator. Define:

$$L(M) = \{ \Pi \mid \Pi \text{ is an abs-exec of } M \}$$

these are sentences over $\mathbf{pow}(\mathbf{Prop})$

Representing φ as an automaton ...

- Let φ be the temporal formula we want to verify.
- Suppose we can construct automaton B_φ that ‘accepts’ exactly those infinite sentences over **pow**(*Prop*) for which φ holds.
- So B_φ is such that :

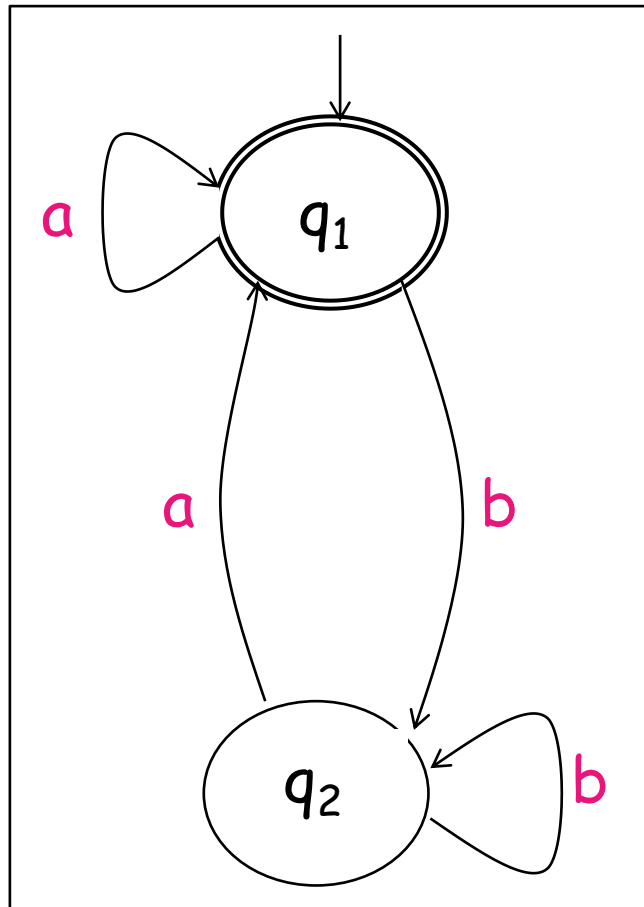
$$L(B_\varphi) = \{ \Pi \mid \Pi \models \varphi \}$$

Re-express as a language problem

- Well, $M \models \varphi$ iff
 - There is no $\Pi \in L(M)$ where φ does not hold.
 - In other words, there is no $\Pi \in L(M)$ that will be accepted by $L(B_{\neg\varphi})$.
- So:

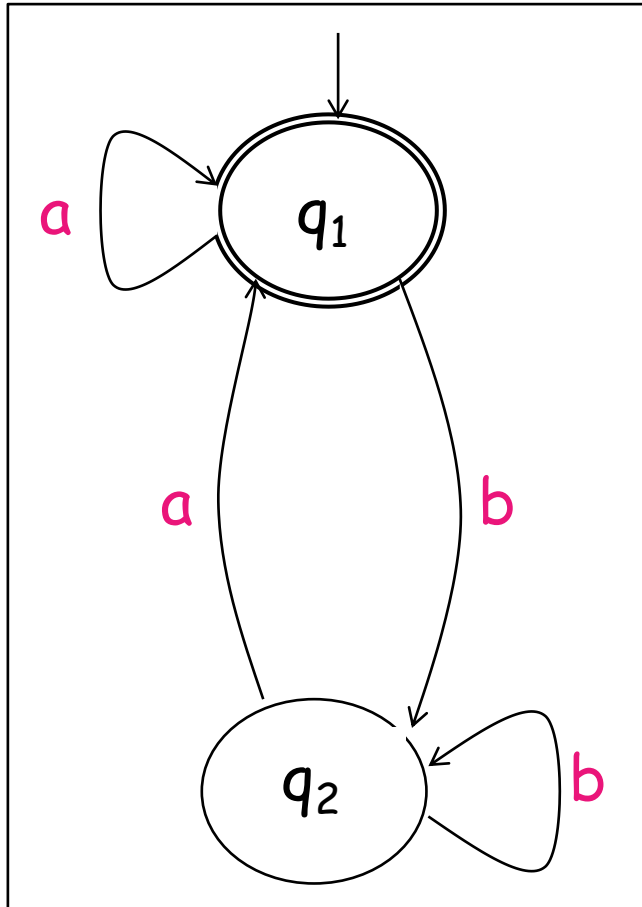
$$M \models \varphi \quad \text{iff} \quad L(M) \cap L(B_{\neg\varphi}) = \emptyset$$

Automaton with “acceptance”



- So far, all paths are accepted. What if we only want to accept some of them?
- Add *acceptance states*.
- Accepted sentence:
“*aba*” and “*aa*” is accepted
“*bb*” is not accepted.
- But this is for finite sentences. For infinite ...?

Buchi Automaton



- Pass an acceptance state infinitely many times.
- Examples

“*ababa*” → not an infinite

“*ababab...*” → accepted

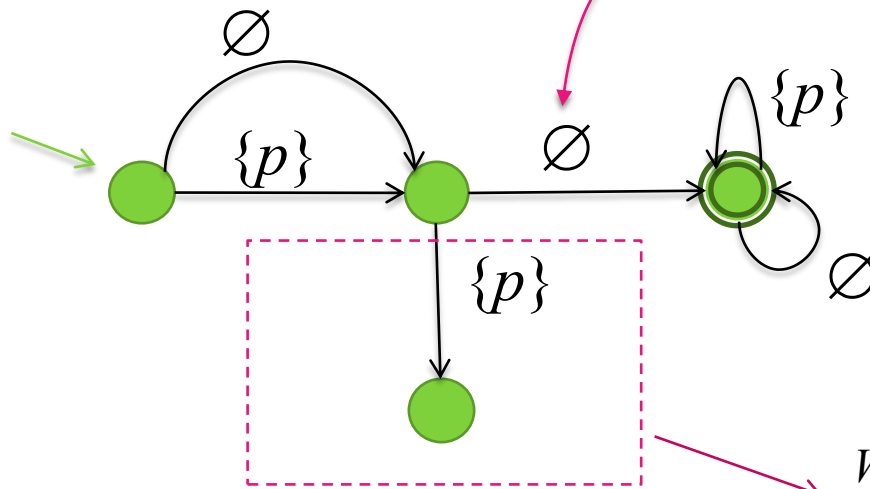
“*abbbb...*” → not accepted!

Expressing temporal formulas as Buchi

Use $\text{pow}(\text{Prop})$ as the alphabet Σ of arrow-labels.

Example: $\neg \mathbf{X}p$ ($= X\neg p$)

We'll take $\text{Prop} = \{ p \}$

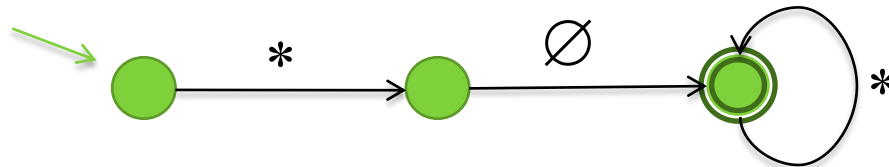


Indirectly saying that p is false.

We can drop this, since we only need to (fully) cover accepted sentences.

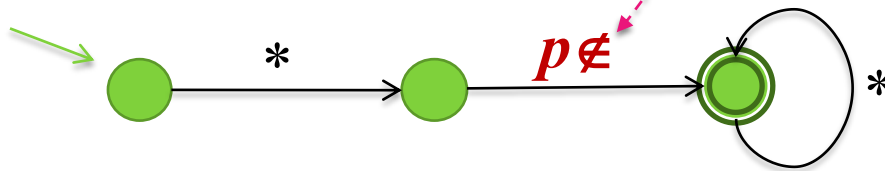
To make the drawing less verbose...

$\neg Xp$, using $Prop = \{p\}$



So we have 4 subsets.

$\neg Xp$, using $Prop = \{p, q\}$



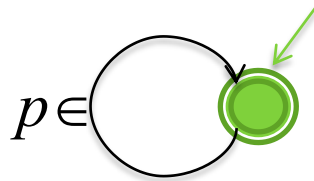
Stands for all subsets of $Prop$ that do not contain p ; thus implying “ p does not hold”.

$p \in$

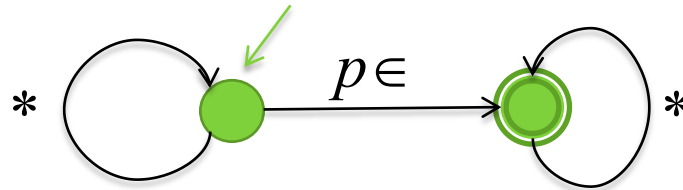
Stands for all subsets of $Prop$ that contain p ; thus implying “ p holds”.

Always and Eventually

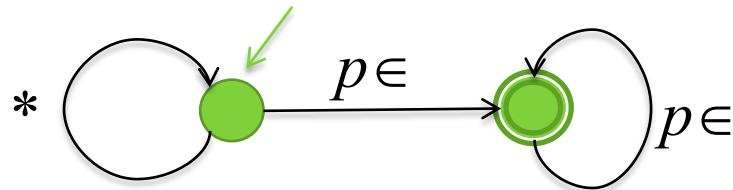
$[]p$



$\diamond p$

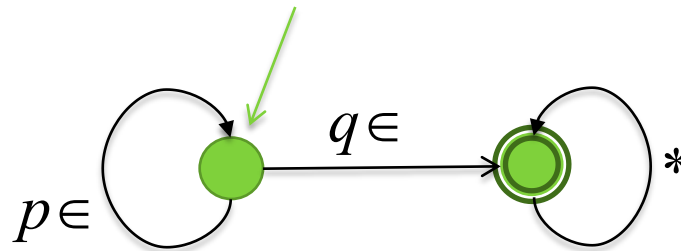


$\diamond []p$

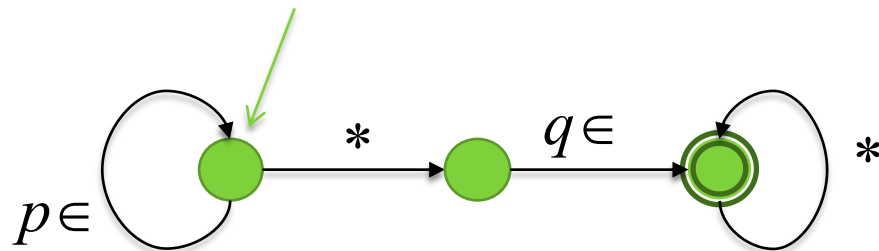


Until

$p \text{ U } q$:



$p \text{ U } \text{X} q$:



Not Until

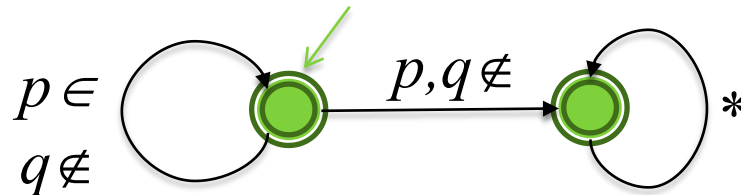
Formula: $\neg (p \textbf{U} q)$

Note first these properties:

$$\neg(p \textbf{U} q) = p \wedge \neg q \textbf{W} \neg p \wedge \neg q \quad \nearrow = \neg q \textbf{W} \neg p \wedge \neg q$$

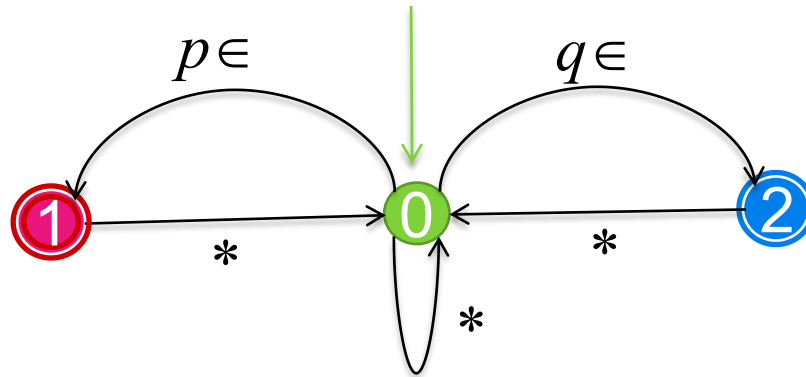
$$\neg(p \textbf{W} q) = p \wedge \neg q \textbf{U} \neg p \wedge \neg q \quad \searrow = \neg q \textbf{U} \neg p \wedge \neg q$$

(also generally when p, q are LTL formulas)



Generalized Buchi Automaton

$$[]\langle\rangle p \quad \wedge \quad []\langle\rangle q$$



Sets of accepting states: $\mathbf{F} = \{ \{1\}, \{2\} \}$

which is different than just $F = \{ 1, 2 \}$ in an ordinary Buchi.

Every GBA can be converted to BA.

Difficult cases

- How about nested formulas like:

$$\begin{array}{l} (\mathbf{X}p) \mathbf{U} \ q \\ (p \mathbf{U} \ q) \mathbf{U} \ r \end{array}$$

Their Buchi is not trivial to construct.

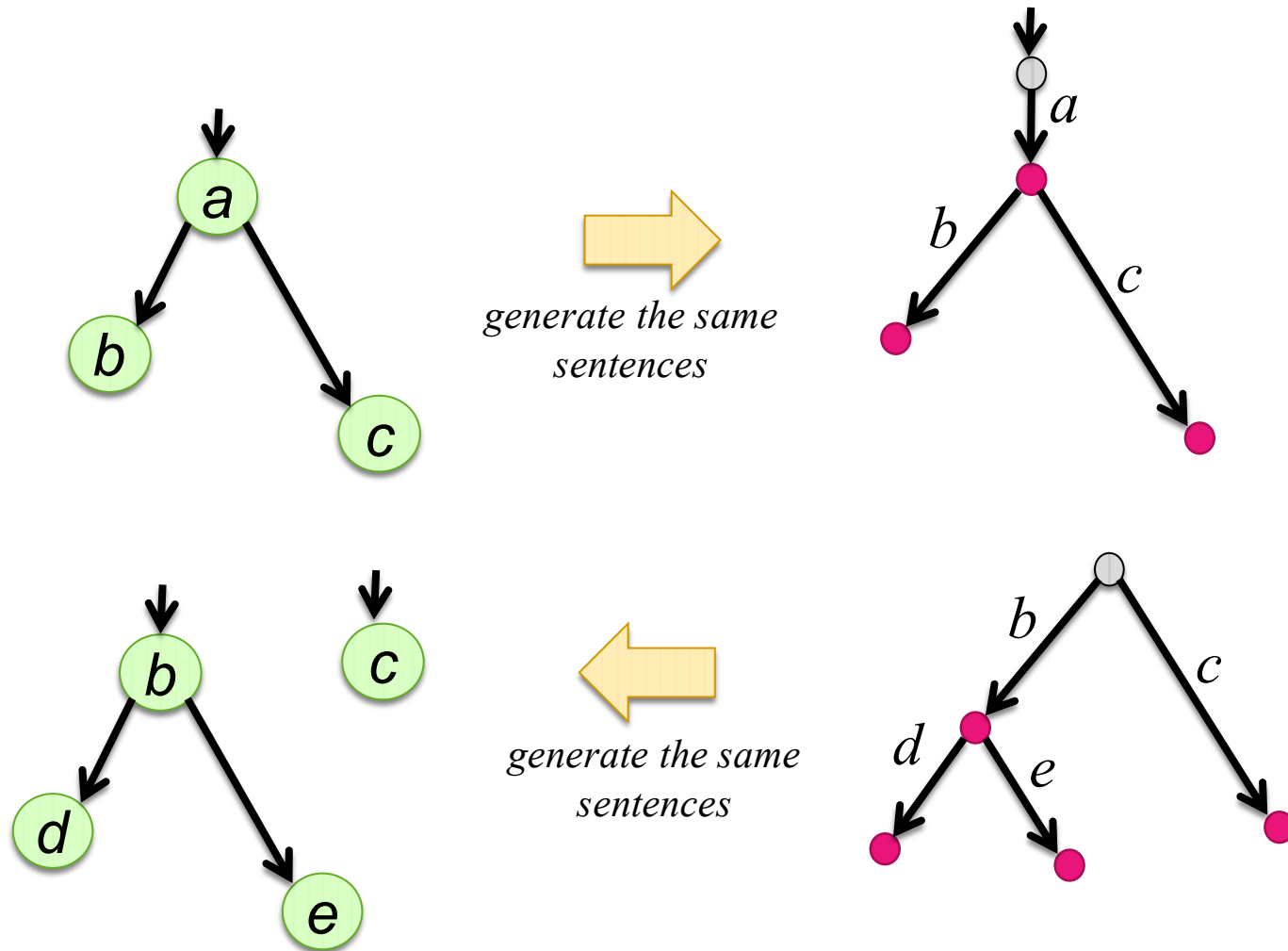
- Still, any LTL formula can be converted to a Buchi. SPIN implements an automated conversion algorithm; unfortunately it is quite complicated.

Check list

$$M \models \varphi \quad \text{iff} \quad L(M) \cap L(B_{\neg\varphi}) = \emptyset$$

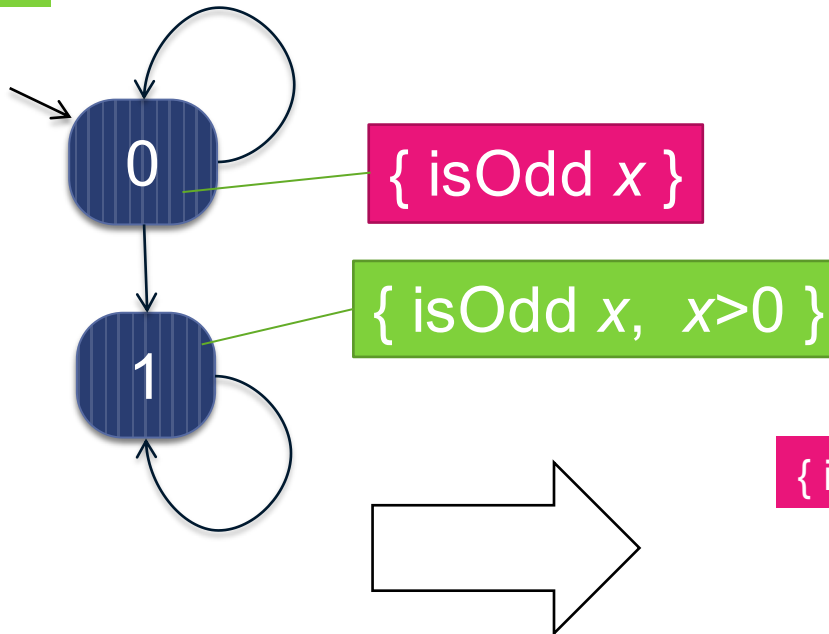
1. How to construct $B_{\neg\varphi}$? $\rightarrow$ Buchi ✓
2. We still have a mismatch, because M is a Kripke structure!
 - Fortunately, we can easily convert it to a Buchi.
3. We still have to construct the intersection.
4. We still to figure out a way to check emptiness.

Label on state or label on arrow...

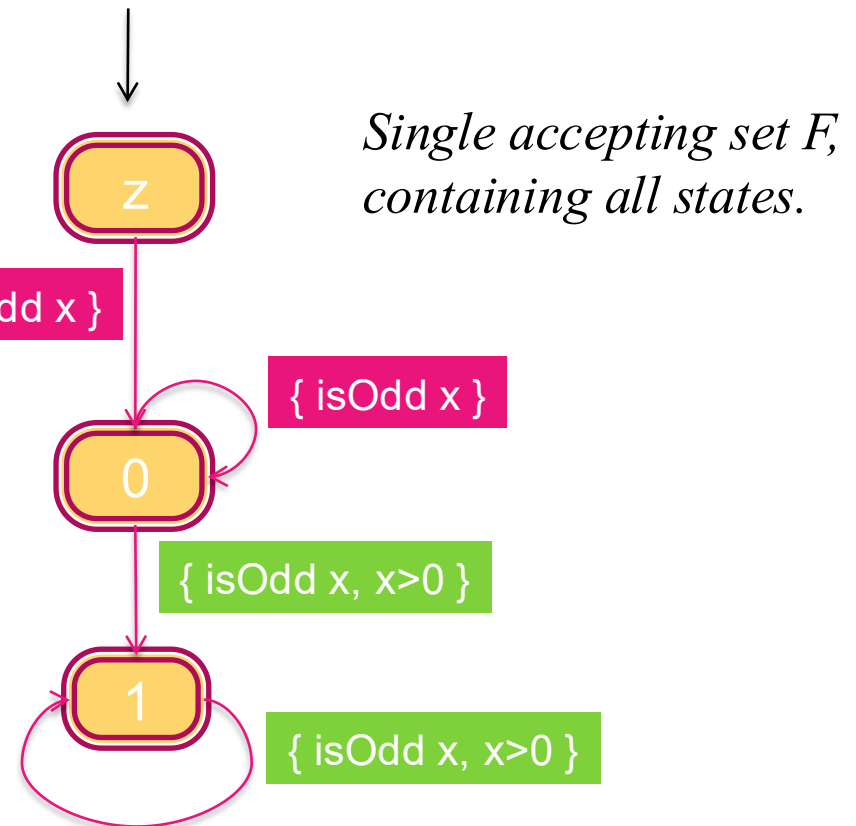


L(M) before and after should remain the same.

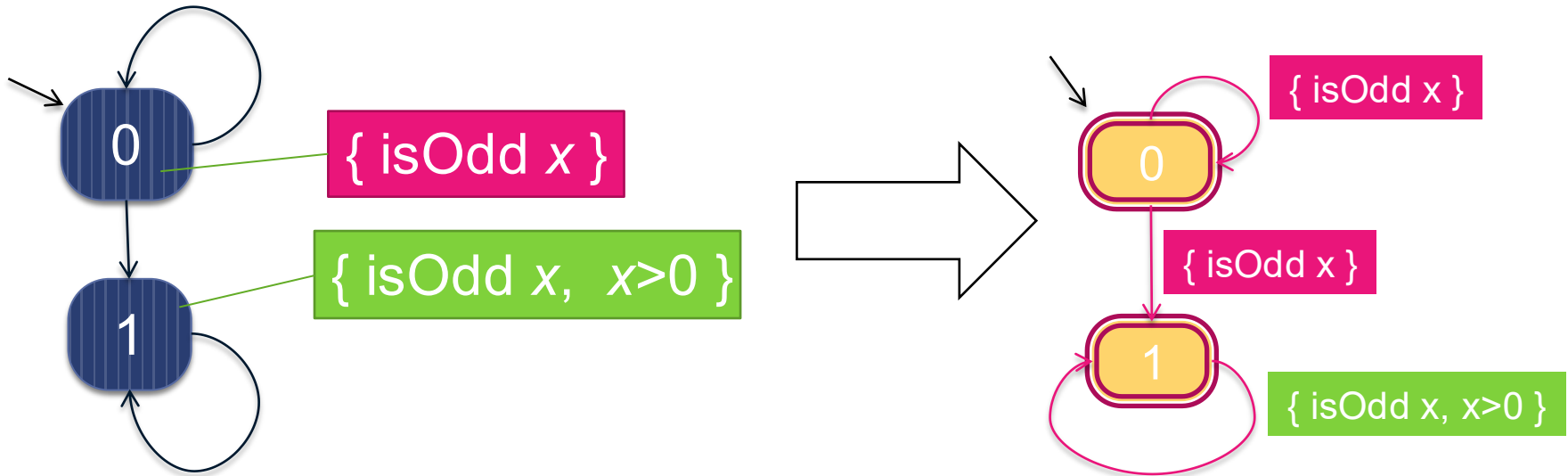
Converting Kripke to Buchi



Generate the same (infinite) sentences!



Converting Kripke to Buchi



Alternatively, since M has no terminal state, we can move the label of state s to all outgoing arrows of s .

Computing intersection

- Rather than directly checking:

The Buchi version of Kripke M
😊

$$L(B_M) \cap L(B_{\neg\varphi}) = \emptyset$$

We check:

$$L(B_M \cap B_{\neg\varphi}) = \emptyset$$

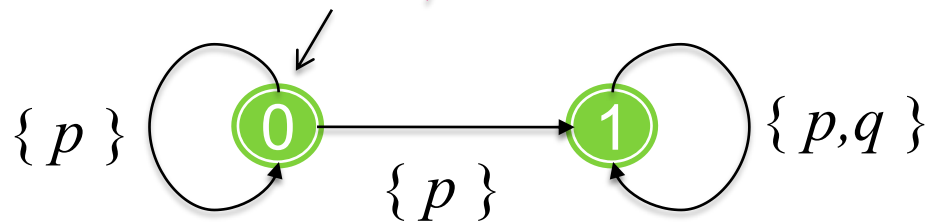
We already discuss this! Execution over such an intersection is also called a “lock-step” execution.

Intersection

- Two buchi automata A and B over the same alphabet
 - The set of states are respectively S_A and S_B .
 - starting at respectively $s0_A$ and $s0_B$.
 - Single accepting set, respectively F_A and F_B .
 - F_A is assumed to be S_A
- $A \cap B$ can be thought as defining lock-step execution of both:
 - The states : $S_A \times S_B$, starting at $(s0_A, s0_B)$
 - Can make a transition only if A and B can *both* make the corresponding transition.
 - A single acceptance set F ; (s, t) is accepting if $t \in F_B$.

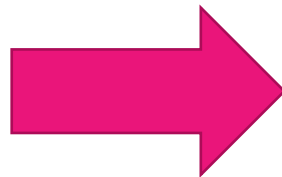
Constructing Intersection, example

B_M :

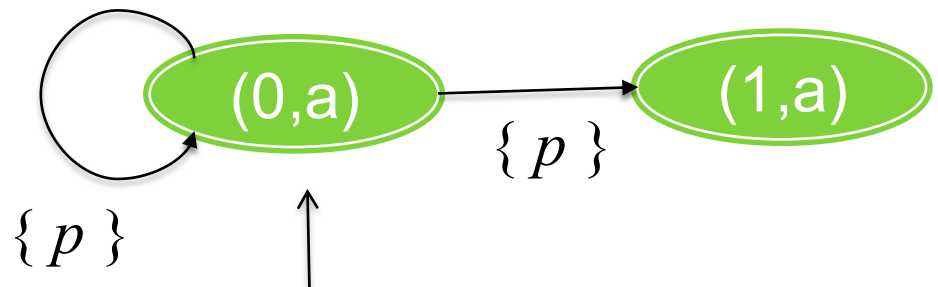


$p : \text{isOdd } x$
 $q : x > 0$

$B_{\neg \langle \rangle q}$:



$B_M \cap B_{\neg \langle \rangle q}$:



Verification

- Sufficient to have an algorithm to check if $L(C) = \emptyset$, for the intersection-automaton C .

$L(C) \neq \emptyset$ iff there is a finite path from C 's initial state to an accepting state f , followed by a cycle back to f .

- So, it comes down to a cycle finding in a finite graph! Solvable.
- The path leading to such a cycle also acts as your counter example!

Approaches

- View $C = B_M \cap B_{\neg\phi}$ as a directed graph.
Approach 1 :
 1. Calculate all strongly connected component (SCCs) of C (e.g. with Tarjan) .
 2. Check if there is an SCC containing an accepting state, reachable from C 's initial state.
- Approach 2, based on Depth First Search (DFS); can be made *lazy* :
 - the full graph is constructed as-we-go, as you search for a cycle.
 - Importantly, if M represents a parallel composition $P_1 \parallel P_2 \parallel \dots$, this means that we can lazily construct B_M .

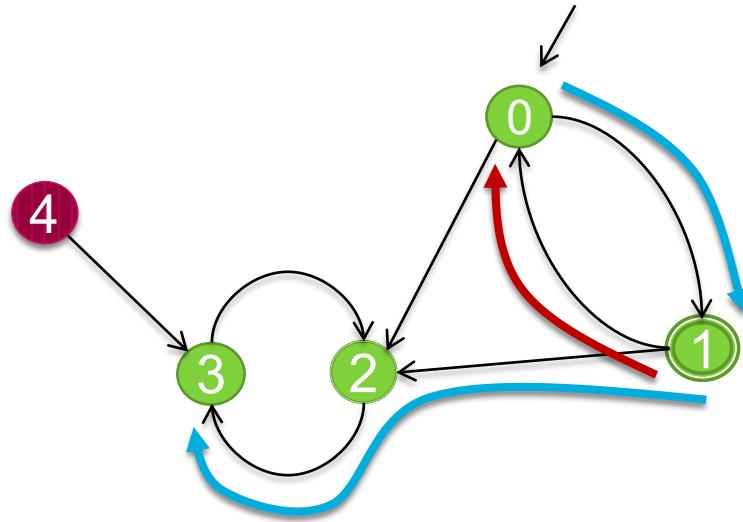
DFS-approach (SPIN)

- DFS is a way to traverse a graph :

```
DFS(u) {  
  
    if (u  $\in$  visited) return ;  
  
    visited.add(u) ;  
  
    for (s  $\in$  next(u)) DFS(s) ;  
  
}
```

- This will visit all reachable nodes. You can already use this to check “state assertions”.

Example



Adding cycle detection

```
DFS(u) {  
    if (u ∈ visited) return ;  
    visited.add(u) ;  
    for each (s ∈ next(u)) {  
        if (u ∈ accept) {  
            visited2 = ∅ ;  
            checkCycle (u,s) ;  
        }  
        DFS(s) ;  
    }  
}
```

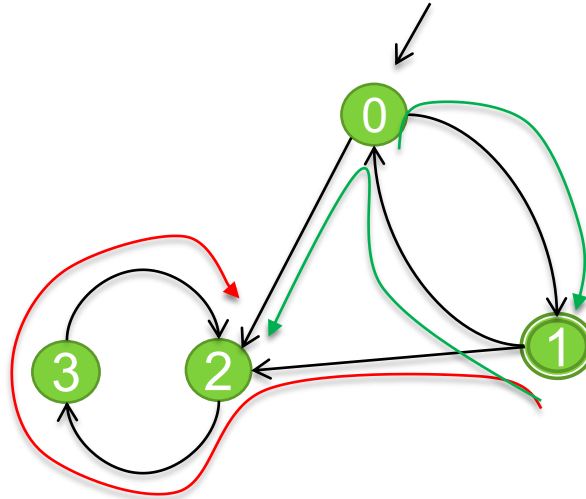
checkCycle is another DFS

```
checkCycle(root, s) {  
  
    if (s = root) throw CycleFound ;  
  
    if ( s ∈ visited2 ) return ;  
    visited2.add(s) ;  
    for each (t ∈ next(s))  
        checkCycle(root, t) ;  
}
```

Can be extended to keep track of the path leading to the cycle → counter example.

See Lecture Notes.

Example



checkCycle(1,2)

root

the node currently being processed

Tweak: lazy model checking

- Remember that automaton to explore is $C = B_M \cap B_{\neg\varphi}$
- In particular, B_M can be huge if $M = P_1 \parallel P_2 \parallel \dots$
- Can we construct C lazily?
- Benefit : if a cycle is found (verification fails), effort is not wasted to first construct the full C .
- Of course if φ turns out to be valid, then C will in the end be fully constructed.
- How to deal with concrete states (rather than abstract states a la Kripke) ?

Lazily constructing the intersection

- Assume first that P is just a single process.
- Only need to change this in the DFS :

for each ($s \in \textit{next}(u)$)
if ($u \in \textit{accept}$)

“**next**” of the intersection
automaton $C = B_M \cap B_{\neg\varphi}$

- Each state of C is of type $S_M \times S_{\neg\varphi}$.
- To check $(s_1, s_2) \in \textit{next}_C(u_1, u_2)$, we check if there is a label L , such that: $s_1 \in \textit{next}_{B_M}(u_1, L) \wedge s_2 \in \textit{next}_{B_{\neg\varphi}}(u_2, L)$
- $(u_1, u_2) \in \textit{accept}_{B_M} \equiv u_2 \in \textit{accept}_{B_{\neg\varphi}}$

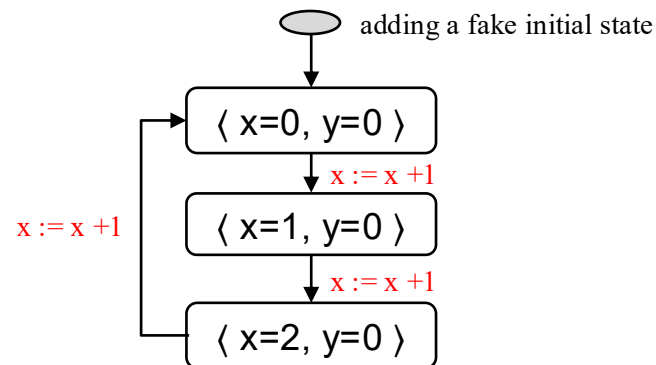
Dealing with concrete states

Consider a concrete program Prg :

var $x = 0$, $y = 0$;

repeat ($x := x+1 \bmod 3$)

- A concrete state of Prg is a vector $\langle x, y \rangle$
- FSM Prg representing the program, in terms of concrete states:



- Given a concrete state s and a predicate p , let $\mathbf{eval}(p,s)$ denote the value of p when evaluated on s (so it is either true or false).
- So, to check if there is a successor of s such that a label $L \subseteq Prop$ holds, we check instead, if there is an (atomic) transition α in Prg such that for all $p \in L$, $\mathbf{eval}(p, \alpha(s))$ is true, and for all $q \notin L$, $\mathbf{eval}(q, \alpha(s))$ is false.

Dealing with concrete states

- So, if M is a program with concrete states, e.g. checking this in the DFS:

$$(s_1, s_2) \in next_C(u_1, u_2)$$

comes down to checking if there is an atomic transition α of M and a label L of $B_{\neg\varphi}$ such that $s_2 \in next_{\neg\varphi}(u_2, L)$, and :

- for all propositions $p \in L$, **eval**(p , $\alpha(u_1)$) = true
- for all propositions $q \in \text{Prop}/L$, **eval**(q , $\alpha(u_1)$) = false

What if $M = P_1 \parallel P_2 \parallel \dots$?

- We discussed the parallel composition of FSAs, e.g. :



Note that here α and β represent actions.

- Literally applying such parallel composition on Buchi automata makes less sense because the labels are properties rather than actions.

We instead construct the parallel M before converting it to Buchi for calculating intersection with $B_{\neg\varphi}$.

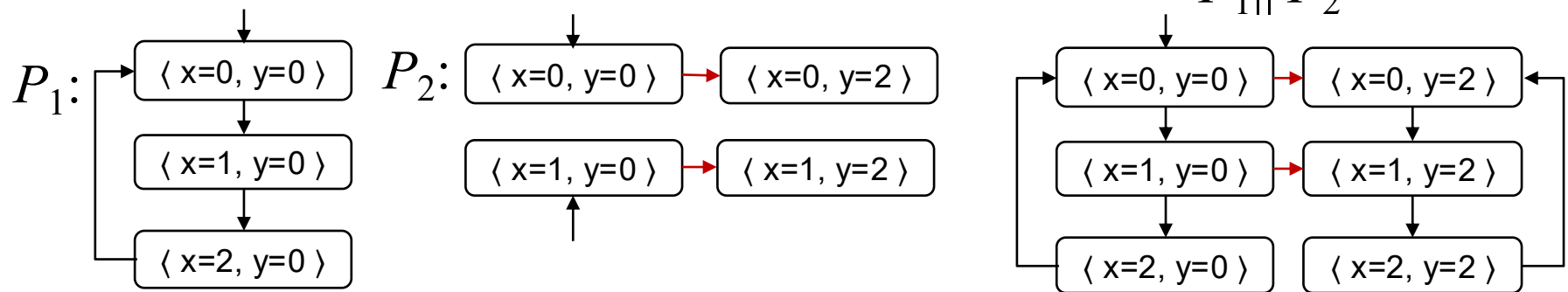
Example constructing $P_1 \parallel P_2$

Consider a concrete program $Prg = P_1 \parallel P_2$:

var $x = 0, y = 0$;

P_1 : **repeat** ($x := x+1 \bmod 3$)

P_2 : ($x \neq 2$) ; $y := 2$



- In the above example, we explicitly construct the concrete state FSM of $P_1 \parallel P_2$
- We can instead construct $P_1 \parallel P_2$ lazily as we construct the intersection automaton with $B_{\neg\varphi}$

What if $M = P_1 \parallel P_2$?

- E.g. checking this in the DFS:

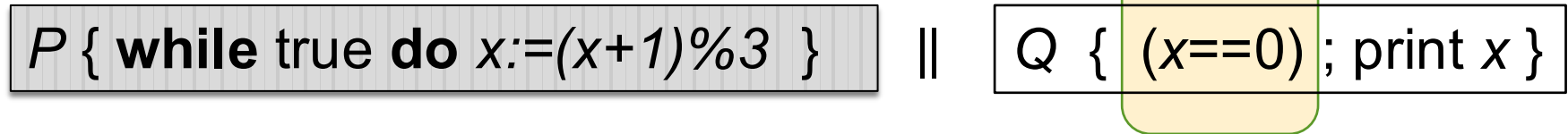
$$(s_1, s_2) \in next_C(u_1, u_2)$$

now comes down to checking if there is an atomic transition α of either P_1 or P_2 , and a label L of $B_{\neg\varphi}$ such that $s_2 \in next_{\neg\varphi}(u_2, L)$, and :

- for all propositions $p \in L$, $\mathbf{eval}(p, \alpha(u_1)) = \text{true}$
- for all propositions $q \in \text{Prop}/L$, $\mathbf{eval}(q, \alpha(u_1)) = \text{false}$

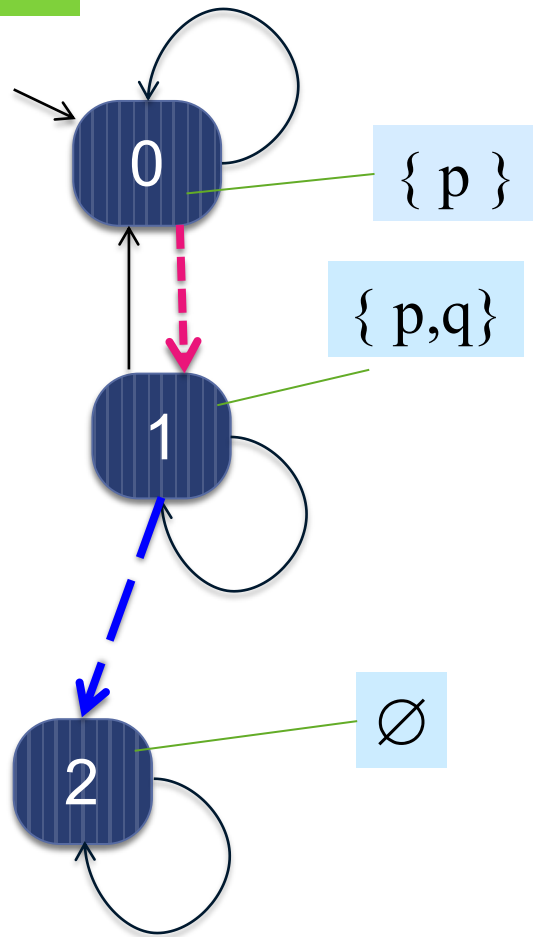
Fairness

Consider this concurrent system :



- In the above system, it is possible that `print x` is ignored forever. Still, we may want to analyze the behavior of the system when it behaves “fairly”.
- We need to first define what we mean by “fair”. Several standard definitions;
 - **Weak fairness** : any action that is persistently enabled will eventually be executed.
 - **Strong fairness** : any action that is kept recurrently enabled (but not necessarily persistently enabled) will eventually be executed.
- Imposing fairness means: when verifying $M \models \varphi$, we only need to verify φ wrt fair executions of M .
 - A **fair execution** : an execution respecting the assumed fairness condition.

Verifying properties under fairness



- If a fairness assumption can be expressed with some LTL formula F , we can instead verify $M \models F \rightarrow \varphi$
- No need for a special algorithm!
- E.g. weak fairness F_1 on the red (short dashed) arrow:
 - $\Box (\Box(p \wedge \neg q) \rightarrow \Diamond(p \wedge q))$
- Strong fairness F_2 on the blue (long dashed) arrow:
 - $\Box (\Box\Diamond(p \wedge q) \rightarrow \Diamond(\neg p \wedge \neg q))$
- Example of property to verify:
 $F_1 \wedge F_2 \rightarrow \Diamond(\neg p \wedge \neg q)$

Conclusion

- We can use FSAs to abstractly model concurrent programs.
- We can use LTL to express run-time properties: safety and progress.
- The model checking algorithm is thorough.
- Rather than FSAs with atomic predicates, you can imagine letting the FSAs to have concrete states.
 - You can then model check real programs.
 - The FSAs could be very large, but we can bound the input domains, and the depth of the search, $\rightarrow$ bounded model checking.
 - Combination with testing: to construct an execution so that M behaves as φ , model-check this: $L(B_M \cap B_\varphi) = \emptyset$